LAB5 Q2: Inventory Log Processor

You are tasked with writing a Python script to help a small bookstore manage its inventory. The store provides you with a daily transaction log file named `transactions.txt`. This file records every time books are received (`receive`) or sold (`sell`).

Your script **must read this file**, process the transactions in order, and then **print the final inventory** levels as **A PYTHON DİCTİONARY**.

The `transactions.txt` file contains one transaction per line. Each line uses a specific format: **`operation:item_name:quantity`**

- **operation:** Either `receive` (adding stock) or `sell` (removing stock).
- **item_name:** The title of the book.
- **quantity:** The number of books involved in the transaction.

Detailed Requirements

1. **Create a Function:** Write a function named `process_inventory` that accepts one argument: `file_name`.
2. **Use a Dictionary:** Inside the function, you must use a dictionary to keep track of the stock. The **keys** will be the item names (book titles), and the **values** will be the current quantity in stock (as an integer).
3. **Read the File:** Your function should open and read the specified file line by line.
4. **Process Operations:**
    - For each line, you must parse it by splitting it at the colon (`:`) character.
    - You should clean up any leading/trailing whitespace from the parts.
    - Convert the `quantity` part to an integer.
    - If the operation is `receive`, you will **add** the quantity to the item's stock.
    - If the operation is `sell`, you must **first check** if there is enough stock.
        - **Sufficient Stock:** If the current stock (or 0 if the item isn't in the dictionary) is *greater than or equal to* the quantity being sold, subtract the quantity from the stock.
        - **Insufficient Stock:** If the quantity to be sold is *greater than* the current stock, the sale must fail. **Do not change the stock level.** You must print a warning message to the console, such as: `SALE FAILED: Tried to sell 50 x '1984', but only 20 in stock.`
5. **Handle Bad Data (Logically):**
    - You can assume the `quantity` will always be a valid number.
    - You should ignore any lines that are blank.
    - You should ignore any lines that do not contain exactly two colons (i.e., are not in the `a:b:c` format).
6. **Final Output:** After processing the entire file, the function must print the final inventory dictionary to the console.

INPUT	OUTPUT
receive:Dune:50	Received: 50 x 'Dune'. New stock: 50
receive:Foundation:30	Received: 30 x 'Foundation'. New stock: 30
sell:Dune:10	Sold: 10 x 'Dune'. Remaining stock: 40
receive:Dune:5	Received: 5 x 'Dune'. New stock: 45
sell:Foundation:35	SALE FAILED: Tried to sell 35 x 'Foundation', but only 30 in stock.
sell:Dune:45	Sold: 45 x 'Dune'. Remaining stock: 0
receive:1984:20	Received: 20 x '1984'. New stock: 20
sell:1984:5	Sold: 5 x '1984'. Remaining stock: 15
invalid_line_format	Skipping malformed line: 'invalid_line_format'
receive:Foundation:10	Received: 10 x 'Foundation'. New stock: 40
sell:Dune:1	SALE FAILED: Tried to sell 1 x 'Dune', but only 0 in stock.
	Final Inventory Report: {'Dune': 0, 'Foundation': 40, '1984': 15}